Data Center–Grid Flex Optimizer — PoC Spec

Nov 6, 2026 · @Nandana

Summary

We will submit a working **optimization and data science product** that models how an AI data center and the grid operate together, hour by hour, and measures the impact on both. It runs on real hourly ERCOT (Texas) grid, weather and emissions data, and answers four questions:

1. **Capacity (data science):** how much new load can the grid host, and when is it tight? Answered with a headroom analysis and an ML day-ahead stress forecast.
2. **Operation (optimization):** how should the data center run each hour to stay within the grid's limit while keeping the most compute? A linear program schedules workloads, battery and backup.
3. **Reliability and design (optimization):** how much flexibility can it reliably deliver, and what battery size and workload mix get there at least cost? Answered by replaying real summers and searching design options.
4. **Impact:** what changes for the grid (peak, capacity, ratepayers), for the data center (compute served, time to power) and for emissions?

The economics of a flexibility agreement are not the product. They are how we justify it and where it goes next (see *Business case and future scope*).

Pitch: "The grid isn't full. It's full for about 40 hours a year. We forecast those hours, optimize how an AI data center rides through them, and measure what that does for the grid, the data center and emissions."

Context

AI is now limited by electricity, not chips. A data center can be built in months, but its grid connection takes years. Grids are planned for their single worst hour, so a data center that draws full power every hour is treated as needing new capacity, even though the grid is below its limit almost all the time.

Flexibility could unlock existing capacity. Duke's Nicholas Institute found 22 large U.S. grids could absorb about 98 GW of new load if it curtailed 0.5% of its yearly energy, including about 10 GW in ERCOT. Average curtailment events last about 2 hours. Texas SB 6 (2025) already requires new loads of 75 MW or more to accept curtailment in emergencies.

Turning that into practice raises three questions:

- **When** is the grid tight? It must be predicted a day ahead, not seen in hindsight.
- **How** should a data center run through those hours? It has to protect customer-facing AI while using batteries, shifting batch work and slowing training.
- **What happens** to grid peaks, reserve capacity, emissions and the data center's compute as a result?

Today these are answered in pieces:

Company	Piece it covers
Emerald AI	Software inside the data center that slows or shifts AI workloads on grid signals
GridCARE	Finds hidden grid capacity for flexible data centers
Camus Energy	Utility software to see and run the grid; "conditional firm" connections

Our angle. One open, transparent system that links all three: forecast the grid, optimize the data center against it, prove it on real history, and measure the impact on each side.

Objective

Objective: a deployed app that, for a chosen grid and data center, forecasts when the grid is tight, optimizes how the data center operates and is designed around those hours, proves it on real history, and reports the impact.

#	Question	Method	What the app returns
Q1	How much load fits, and when is the grid tight?	Headroom analysis on history; ML day-ahead forecast	MW connectable per % curtailed; stress hours a day ahead; events and hours per year
Q2	How should the data center run each hour?	Linear program over workloads, battery and backup	Hour-by-hour schedule; where each MW of cut came from
Q3	How much can it reliably deliver, and how should it be built?	Day-by-day replay of real summers; search over battery size and workload mix	Reliable MW in 95% of events; cheapest design that meets the target; trade-off frontier

#	Question	Method	What the app returns
Q4	What is the impact?	Impact model on replay results	Grid: peak and capacity avoided. Data center: compute served, years earlier online. Emissions: net CO_2. Ratepayers: avoided cost per MWh

Acceptance tests

All tests run on data the models never saw: the held-out test summer, June to September 2025.

ID	What we prove	Pass threshold
A1	The grid is tight for only a few hours	Headroom at 0.5% curtailment is the same order of magnitude as Duke's ~10 GW for ERCOT
A2	Stress hours are predictable a day ahead	Recall ≥ 90% of stress hours; precision reported
A3	Our forecast is worth having	Beats "same hour last week"; MAPE within ~1 point of ERCOT's own forecast
A4	It catches the real system peaks	All four 2025 4CP peak days (19 Jun, 30 Jul, 18 Aug, 4 Sep) are flagged a day ahead
A5	The reliable-MW estimate holds	Reliable MW estimated on summer 2024 is delivered in ≥ 95% of real events in summer 2025
A6	The grid stays safe	Replay: 0 hours where grid load exceeds the planning peak after battery, throttling and backup
A7	AI service is protected	Real-time inference served in 100% of hours
A8	The design search works	Returns the smallest battery meeting the 95% target, and it is cheaper than the default design
A9	The numbers reconcile	Every replay hour balances: compute served + deferred + lost = compute demanded (± 0.1%)

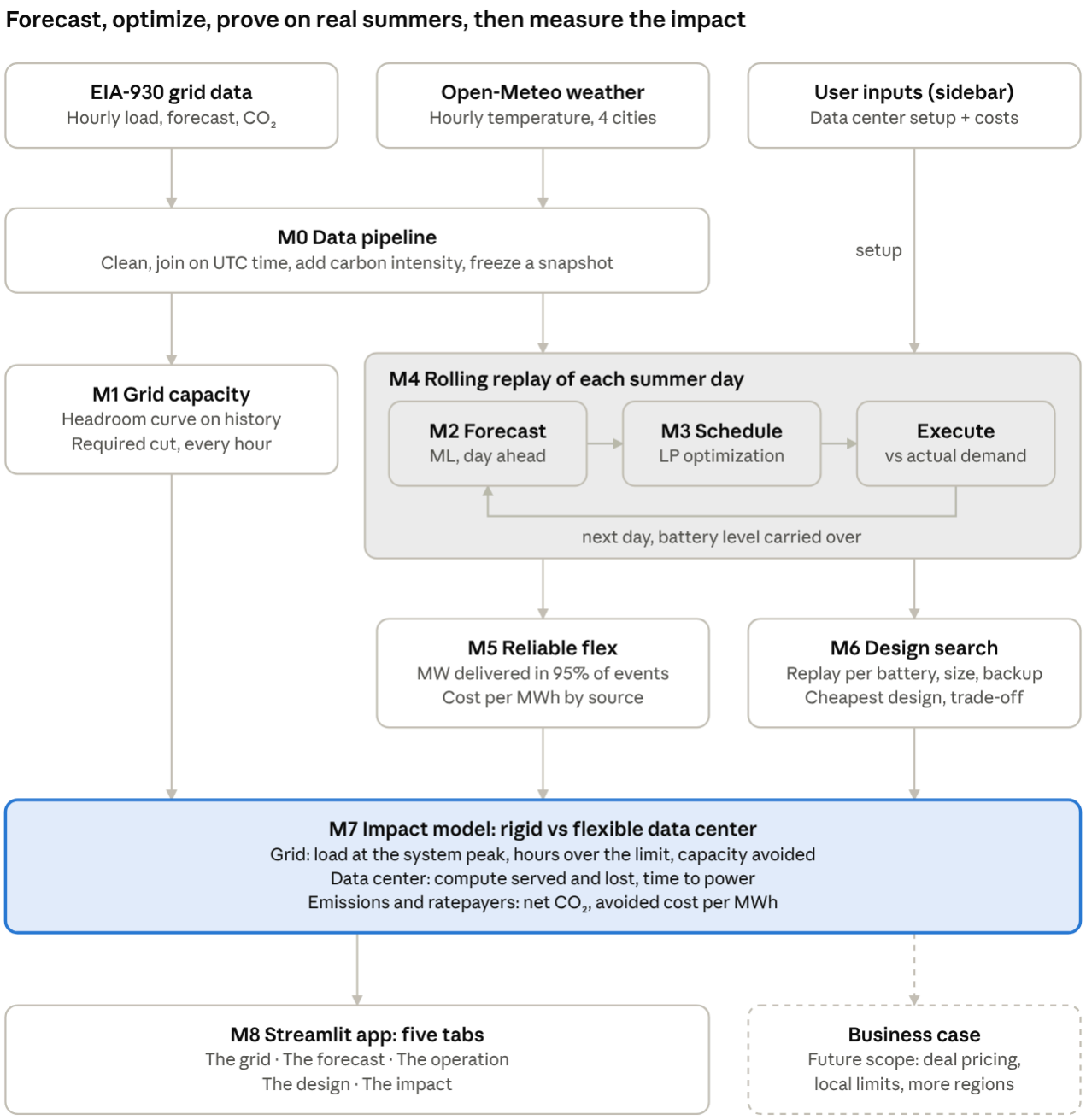
ID	What we prove	Pass threshold
A10	It works for a judge	Public URL; loads in < 15 s; sliders respond in < 3 s; runs offline from a frozen snapshot

How it ties together

Forecast, optimize, prove, then measure.

1. Real data gives the grid's capacity and its tight hours (M1, M2).
2. A daily loop over real summers forecasts, schedules and checks against what actually happened (M2–M4). That shows what the data center can reliably deliver (M5).
3. The same loop, rerun for many designs, finds the cheapest way to build it (M6).
4. The impact model (M7) compares a rigid and a flexible data center for the grid, the data center, emissions and ratepayers.

The business case sits on top as justification and future scope.



system flow · data to impact, 9 modules

Each arrow is a saved file, so each teammate can build against a placeholder version of their input and connect the real one later.

How the data center works with the grid

The data center treats the grid's limit as a cap that moves hour by hour. A day ahead, it gets a forecast of tight hours and plans its workloads around them. On the day, it responds if the grid turns out tighter than forecast. Afterwards, every event is logged and checked.

What each workload can do

An AI data center is not one load. It is several workloads with different flexibility, and the optimizer treats each one differently.

Workload	Default share	Can it flex?	How it flexes	Typical response	Cost of flexing
Real-time inference (chat, APIs)	20% of IT	**No**	Always served at full power	—	—
Batch inference and fine-tuning	30% of IT	Yes, in time	Pause the job queue during the peak; catch up later on idle GPUs, within 24 hours	Minutes	~$0 while idle GPU capacity exists
Training	50% of IT	Yes, in depth	Lower GPU power caps to slow it, or pause at a checkpoint	Seconds to minutes	Compute lost or delayed (default $1,000 per MWh)
On-site battery	100 MW × 4 h	Yes	Discharges to cover part of the facility's draw	Seconds	Wear (default $20 per MWh)
Backup generation	Up to 150 MW	Yes, last resort	Runs on-site gas	About 10 minutes	Fuel (default $150 per MWh) plus CO_2
Cooling	Scales with IT	Automatically	Every IT MW cut also saves cooling, so the grid sees IT cut × PUE	Follows IT	—

Response times are planning assumptions, not measured values. Moving work to another data center on a different grid is future scope.

The signals between grid and data center

When	Grid side	Data center side	Module
Day before	Forecast of tomorrow's stress hours and the cut needed, with a probability for each hour	Plans the next 24 hours: which batch jobs move, when the battery charges, how much training may slow	M2 → M3
During the day	Actual required cut each hour (in practice, the grid operator's dispatch or emergency signal)	Covers any gap beyond the plan in cost order, keeping real-time inference whole	M4
After each event	Measured grid load	Delivered MW logged against the MW asked, by source	M4 → M5

A peak day, step by step

This is an illustration of how the schedule behaves, not a result.

1. **Day before, evening:** the forecast flags 3 pm to 7 pm tomorrow as tight, with the largest cut needed around 5 pm.
2. **Overnight and morning:** the battery is topped up. Batch jobs are pulled forward and run in the morning while the grid has room.
3. **3 pm to 7 pm:** the cut stacks up as the peak builds:
   a. the batch queue pauses
   b. the battery discharges
   c. backup starts if the cut is still short
   d. training GPUs are power-capped only at the very top of the peak

   Real-time inference runs at full power throughout.
4. **The grid tightens more than forecast:** the execution layer cuts training a little deeper or runs backup harder for those hours, and logs the extra.
5. **Evening and night:** training returns to full power. The battery recharges, and paused batch work catches up before its 24-hour deadline.

The order in step 3 follows the default costs. The optimizer chooses by cost, so the order changes with the sliders. For example, a carbon price above zero makes backup more expensive and the optimizer cuts training sooner instead.

Input data

Grid and weather data are real and free. Data center and price settings are user inputs with stated defaults, because no public data exists for individual data centers or their contracts.

Region: ERCOT (Texas)

We use ERCOT, EIA code `ERCO`, because:

- It is one mostly islanded grid that peaks in summer heat, which gives a clean signal for the forecast.
- It is where rules for data centers on the grid are furthest along: SB 6 curtailment, 4CP peaks, demand response programs.
- Duke published an ERCOT figure we can check against.

Fallback: PJM (`PJM`). The pipeline is the same; only the code and weather cities change.

Datasets

Dataset	Source and access	Fields used	Period	Used for
Hourly demand	EIA-930: API v2 (free key) or six-month BALANCE CSVs	UTC time, `Demand (MW)`	Jan 2019 – Sep 2025	Headroom, forecast target, replay, 4CP check
ERCOT's day-ahead forecast	EIA-930, same files	`Demand Forecast (MW)`	Same	Benchmark for test A3
Hourly generation by fuel and CO_2	EIA-930 (available from July 2018)	Net generation by source; estimated CO_2	Same	Hourly grid carbon intensity for the emissions impact
Hourly temperature	Open-Meteo Historical Weather API (no key)	`temperature_2m` for DFW, Houston, San Antonio, Austin, weighted 0.39 / 0.36 / 0.13 / 0.12	Same	Forecast features
Hourly prices (stretch)	ERCOT hub prices via the `gridstatus` Python library	Real-time price, $/MWh	Test summer	Price-aware scheduling; energy cost impact
Backup data source	PUDL cleaned EIA-930	Same	Same	If the EIA API fails

Data cleaning:

- Use UTC timestamps.
- Drop 10–20 Feb 2021 (Winter Storm Uri outages).
- Drop zeros and outliers; fill gaps of 3 hours or less.
- Freeze one snapshot file so results don't shift when EIA revises data.

4CP proxy: 4CP uses 15-minute intervals, but EIA-930 is hourly. We use the hour containing each month's peak, which is close enough to test whether a strategy would have caught the peak.

Data center setup (sidebar)

Input	Default	Note
IT load	1,000 MW	A large AI campus
PUE	1.2	Typical for new hyperscale builds
Real-time inference (never cut)	20%	Assumption
Batch and fine-tuning (shifts within 24 h)	30%	Assumption
Training (can slow or pause)	50%	Remainder
Battery	100 MW × 4 h	10% of IT load
Backup generation	150 MW	On-site gas or similar; last resort
Flex promised to the grid	100 MW	What the operator offers; the app tests whether it holds
Reliability target	95% of events	How often the promise must be fully delivered

Cost and impact parameters (sidebar)

These turn MWh into $ and tonnes, so the optimizer can rank options and the impact model can report results. Every figure is editable; defaults are rough starting points, not forecasts.

Input	Default	Used in	Basis
Value of compute	$1,000 per IT MWh	Scheduler objective; impact	Rough GPU-rental equivalent
Battery wear	$20 per MWh cycled	Scheduler; supply curve	Assumption
Battery capital cost	$40 per kWh-year (annualized)	Design search	Assumption
Backup fuel and maintenance	$150 per MWh	Scheduler; supply curve	Assumption

Input	Default	Used in	Basis
Backup emissions	0.5 t CO_2 per MWh	Emissions impact	Assumption for on-site gas
Carbon price (optional)	$0 per t CO_2	Carbon-aware scheduling	Set above $0 to make the optimizer avoid dirty hours
Grid's avoided capacity cost	$120 per kW-year	Grid impact	Reference: PJM 2026/27 capacity price of $329.17 per MW-day
Wait for a firm vs flexible connection	60 vs 12 months	Data center impact	User inputs, not forecasts

Output data

Each module writes one file. The app reads only these files, so it loads fast and still runs if APIs are down.

File	Written by	Columns or keys
`data/grid_hourly.parquet`	M0 Data pipeline	`ts_utc`, `load_mw`, `ba_forecast_mw`, `temp_c`, `co2_t_per_mwh`, `is_4cp`, `excluded`
`out/headroom.csv`	M1 Grid capacity	`new_load_mw`, `curtail_hours`, `curtail_pct_energy`, `avg_event_hours`, `max_cut_mw`
`out/forecast_test.csv`	M2 Forecaster	`ts_utc`, `actual_mw`, `model_mw`, `ba_forecast_mw`, `naive_mw`, `p_stress`, `stress_flag`
`out/metrics.json`	M2 Forecaster	MAPE for all three forecasts; recall; precision; 4CP days caught
`out/replay_hourly.csv`	M4 Replay	`ts_utc`, `required_cut_mw`, `realtime_mw`, `batch_mw`, `training_mw`, `batt_mw`, `soc_mwh`, `backup_mw`, `dc_draw_mw`, `grid_total_mw`, `over_limit_mw`

File	Written by	Columns or keys
`out/events.csv`	M4 Replay	`event_id`, `start_utc`, `hours`, `asked_mw`, `delivered_mw`, MW from batch, battery, backup and training, `shortfall_mw`
`out/reliability.json`	M5 Reliable flex	Reliable MW at the target; share of events fully delivered; MW by source
`out/supply_curve.csv`	M5 Reliable flex	`source`, `mw`, `cost_per_mwh`, `energy_limit_mwh`
`out/scenarios.parquet`	M6 Design search	One row per design: size, battery, mix → reliable MW, compute lost %, backup MWh, CO_2, cost
`out/impact.json`	M7 Impact model	Grid, data center, emissions and ratepayer metrics, rigid vs flexible

What the judge sees

Headline numbers on every screen:

- MW connectable without new capacity
- reliable MW delivered
- compute lost %
- net CO_2 change

Five screens:

1. **The grid:** headroom curve, plus the hours and MW this data center would need to cut.
2. **The forecast:** test-summer forecast vs actual, with stress hours and the four 4CP days marked, and a scorecard against ERCOT's forecast.
3. **The operation:** peak-week schedule with workloads stacked under the grid limit; total grid load with a rigid vs a flexible data center; delivered MW per event against the promise line.
4. **The design:** supply curve (MW at each cost per MWh); the frontier of MW connectable vs compute lost across designs, with the cheapest design that meets the target highlighted.
5. **The impact:** grid, data center, emissions and ratepayer scorecards, each rigid vs flexible.

Detailed specs

Nine modules. Heavy steps (download, training, the full-summer replay) run once offline in `build.py`. The app reruns only the fast parts (one-week schedule, supply curve, impact numbers) live when sliders move.

Shared definitions

- **L_t:** actual ERCOT demand in hour t (MW).
- **P:** planning peak, the highest hourly demand from 2022 to 2024.
- **F:** facility load = IT MW × PUE.
- **Required cut** in hour t: $c_t = \min(\max(L_t + F - P, 0), F)$. This is how much the data center must drop so the grid stays at or under P.
- **Event:** a run of consecutive hours with $c_t > 0$.
- **K:** the flex the operator promises, in MW.

Data splits

Purpose	Train on	Evaluate on
Tuning (model settings, reliable MW estimate)	Jan 2019 – May 2024	Jun – Sep 2024
Final test, reported to judges	Jan 2019 – May 2025	Jun – Sep 2025

M0. Data pipeline (`pipeline.py`)

1. **Demand:** download types `D` and `DF` for `ERCO` from the EIA API v2 route `electricity/rto/region-data`, paging 5,000 rows per call. If the API fails, read the six-month CSVs.
2. **Weather:** download hourly temperature for the four cities in UTC and take the weighted mean.
3. **Clean and save:** apply the cleaning rules, join on UTC time, mark the hour of each month's peak in June–September as `is_4cp`, and write `grid_hourly.parquet`.
4. **Check:** about 59,000 rows, no duplicate hours, under 1% excluded.

M1. Grid capacity (`grid_capacity.py`)

1. **Headroom curve.** For new load X from 0 to 25,000 MW in 250 MW steps, use 2022–2024 demand to count hours with a required cut, the share of energy cut, average event length and the largest cut. This is Duke's method; check it against Duke's ERCOT figure (test A1).
2. **The ask for this data center.** At the chosen F, report:

- events per year and hours per year
- the longest event
- **K_ask**, the largest required cut

K_ask is the flexibility the grid needs from this data center.

M2. Stress forecaster (`forecast.py`)

Task: at the end of day D, forecast hourly demand for day D+1 using only data available by then.

Features:

- **Calendar:** hour, day of week, US holidays, and day of year as sine and cosine.
- **Weather:** temperature; cooling degrees, max(T − 20 °C, 0); heating degrees, max(10 °C − T, 0); the day's maximum temperature; the 3-day mean temperature.
- **Lagged load:** lag 24 h, lag 168 h, and the previous day's peak.

Model: a linear regression on lags and degree-hours, so it can extrapolate to record heat, plus gradient-boosted trees fitted on what the linear part misses.

Stress probability:

```
p_t = 1 - \Phi\left(\frac{(P - F) - \hat L_t}{\sigma_{h(t)}}\right)
```

- σ is the forecast error spread for that hour of day, measured on the tuning summer.
- Flag the hour when $p_t \geq \tau$. Choose τ on the tuning summer as the highest value that still catches at least 90% of stress hours.

Benchmarks: ERCOT's own forecast (`DF`) and a naive forecast (same hour last week).

Metrics:

- MAPE overall, and on the top 5% of demand hours
- recall and precision on stress hours
- how many of the four 4CP days were flagged a day ahead

M3. Workload scheduler (`scheduler.py`)

A linear program solved with SciPy HiGHS or PuLP, over 24 hours (replay) or 168 hours (app week view).

Decision variables, per hour:

- training p_t
- batch b_t
- battery charge and discharge

- battery state of charge s_t
- backup u_t

Objective: minimise the total cost of flexing over the horizon, where w is battery wear and f is backup fuel. Lost training compute is valued at v ($/MWh); optional carbon price λ applies to grid draw at hourly grid intensity e_t and to backup at its emission factor e_b:

```
\min \sum_t \Big[ v\,(\bar p - p_t) + w\,(c_t + d_t) + f\,u_t + \lambda\,
(e_t\, g_t + e_b\, u_t) \Big]
```

Here $\bar{p}$ is the training load at full power and g_t is grid draw. With λ = 0 this simply keeps the most compute; with λ > 0 the optimizer also prefers to move batch work into cleaner hours.

Constraints, every hour:

- **Grid limit:** facility draw minus battery discharge minus backup ⩽ F − $\hat{c}_t$, where $\hat{c}_t$ is the planned required cut from the forecast plus a safety buffer.
- **GPU capacity:** real-time + batch + training ⩽ IT capacity. Real-time inference is fixed and never cut.
- **Batch deadline:** each day's batch work is delivered within 24 hours.
- **Battery:** state of charge carries hour to hour at 90% charging efficiency and stays within its energy and power limits. It ends each day at least half full.

M4. Rolling replay (`replay.py`)

This module ties the system together and supplies most of the proof. For each day of the test summer:

1. **Forecast:** at the end of day D, forecast D+1 (M2).
2. **Plan:** schedule D+1 on the forecast (M3), starting from the battery level left at the end of D.
3. **Execute against reality:** apply the actual required cut c_t. When the plan falls short, cover the gap in cost order:
   a. batch shift
   b. battery
   c. backup, up to its MW cap
   d. training throttle

   Whatever remains is logged as `over_limit_mw`.
4. **Log:** record each hour's sources and each event's asked MW, delivered MW and shortfall. Carry the battery level into the next day.

Comparison runs:

- a rigid data center (no flexing)
- perfect foresight, as the best possible case
- ±2 °C weather noise, as a stand-in for weather forecast error

M5. Reliable flex and supply curve (`flex_supply.py`)

Capability per event. For each event in the tuning summer, solve a small LP: the largest constant cut the data center can hold for the whole event. It starts from the real battery level and batch backlog at that moment, so back-to-back heat-wave days count against it.

Reliable MW:

- **K_rel** = the 5th percentile of capability across events (with the 95% target).
- **Promise check:** K_rel ≥ K_ask means the promise holds. Otherwise, report the largest F where it does.
- Re-test on the test summer (test A5).

Supply curve. Sources are ordered by cost per MWh of flex:

Source	MW available	Limit	Cost per MWh
Batch shift	Batch MW × PUE	Idle GPU capacity to catch up, (1 − utilisation) × IT MW, within 24 h	~$0
Battery	Battery MW	Battery energy, refilled between events	Wear + round-trip losses × energy price
Backup generation	Backup MW cap	Fuel; permit hours (stretch)	Fuel and maintenance
Training throttle	Training MW × PUE	None, but lost compute	Value of compute
Real-time inference	0	Never cut	—

With the defaults, cheap flex (batch and battery) runs out first and training is the most expensive source. The curve shows exactly where that happens.

M6. Design search (`design.py`)

The scheduler (M3) optimizes how the data center **runs**. This module optimizes how it is **built**.

Design variables:

Variable	Search range	Steps
Facility size F	500 – 3,000 MW	6
Battery power	0 – 30% of IT MW	6
Battery duration	0, 2, 4, 6, 8 h	5
Backup cap	0, 10%, 20% of F	3

The workload mix is a scenario (set by the tenant), not a decision.

Method:

1. Run the replay (M4) for every design on the tuning summer: about 540 runs, each roughly 122 one-day LPs. Run it offline in parallel.
2. For each design, record:
   - reliable MW
   - compute lost %
   - backup MWh
   - CO_2
   - yearly cost = battery capital + compute lost × v + wear + backup fuel
3. **Cheapest feasible design:** the lowest yearly cost subject to reliability ≥ target and zero hours over the limit.
4. **Frontier:** for each size F, the lowest compute lost % reachable. This is the trade-off a planner reads: "connect 2 GW at 0.3% compute lost, or 3 GW at 1.1%."
5. Confirm the chosen design on the test summer (test A8).

Stretch: replace the grid search with a mixed-integer program that picks battery size and schedule together.

M7. Impact model (`impact.py`)

Each metric is computed twice from the replay, once for a rigid data center (always at full power) and once for the flexible one, and reported side by side.

Grid:

- **Data center load at the system peak hour:** rigid = F; flexible = F minus its cut.
- **Hours the grid exceeds its planning peak:** rigid vs flexible.
- **Firm capacity avoided:** the difference in peak contribution, in MW, and × $120 per kW-year in $.
- **Headroom unlocked:** MW connectable at the chosen curtailment level (from M1).

Data center:

- **Compute:** served, deferred and lost, as MWh and %.
- **Resource use:** battery cycles; backup hours and MWh.
- **Time to power:** months earlier online (from inputs), turned into extra compute MWh.

Emissions:

- **Grid intensity:** hourly CO_2 ÷ net generation for ERCOT, from EIA-930.
- **Net CO_2:** Σ grid draw × intensity, plus backup MWh × 0.5 t, for each case. Reported as the net change: cleaner timing of batch work against dirtier backup hours.

Ratepayers:

- **Avoided capacity cost per MWh:** avoided capacity $ ÷ ERCOT's annual load in MWh (from EIA-930), i.e. avoided cost per MWh of everyone's electricity.
- **4CP cost shift:** the data center's load cut in the four 4CP hours. This lowers its transmission bill, but shifts that cost to other customers. We report it openly as a side effect regulators are reviewing.

M8. App (`app.py`, Streamlit)

Area	Contents
Sidebar	Region; data center setup; cost and impact parameters; reliability target; carbon price
Header	Four headline numbers
Tab 1 · The grid	Headroom curve; this data center's required cuts
Tab 2 · The forecast	Forecast vs actual; 4CP days; scorecard against ERCOT and naive
Tab 3 · The operation	Peak-week schedule; rigid vs flexible grid load; delivered MW per event
Tab 4 · The design	Supply curve; trade-off frontier; cheapest feasible design
Tab 5 · The impact	Grid, data center, emissions and ratepayer scorecards; a short business case panel

Repo layout: `data/` (frozen snapshot), `out/`, `src/` (M0–M7), `build.py`, `app.py`, `requirements.txt`, `README.md`.

Deployment: Streamlit Community Cloud from a public GitHub repo. Fallback: run locally, plus a recorded video.

Assumptions and limitations

State these in the demo before judges raise them.

Limitation	Why it matters	How we handle it
Grid-wide view only	Real connections also hit local substation and line limits	Present results as an upper bound; local studies come next
Average, not marginal, emissions	Peak hours are often served by gas peakers, so average intensity understates the carbon saved by flexing	Label it; marginal emissions are future scope
Actual temperatures stand in for weather forecasts	Overstates forecast accuracy	Report the ±2 °C noise run beside the main result
Cost defaults are rough	Compute value, wear and fuel costs shift which source the optimizer picks	Every figure is a slider; the design search reruns offline for new defaults
Compute scales linearly with power	GPU throughput at lower power isn't exactly proportional	Labelled as a simplification
Backup generation limits	Air permits cap run-hours	Backup cap is a design variable; permit hours are a stretch goal
Design search is a grid, not a full optimization	It may miss the exact best design between grid points	Fine steps near the optimum; a mixed-integer program is a stretch goal
Will operators actually curtail?	Market monitors doubt it in real emergencies	Reliable MW (test A5) is sized to what was actually deliverable in real events

Business case and future scope

This is how we justify the product, and where it goes next. It is not the core of the PoC. The optimizer shows a data center *can* flex at little cost. The next question a business would ask is why it *would*.

Why a data center would flex:

Incentive	Why it matters	Real example in ERCOT
Get connected sooner	Starts earning on its compute years earlier	Flexible connections (GridCARE, Camus)
Avoid peak transmission charges	Cutting load in a few peak intervals cuts a year of charges	4CP, worth up to about $50,000 per MW per year
Get paid to stand by	Capacity and performance payments	Emergency Response Service and ancillary services
Avoid forced shutdowns	Planned flexing beats an emergency disconnect	SB 6 (2025)

What the PoC already shows to back this up:

- the compute lost is small (impact model)
- the reliable MW is known (replay)
- the grid's avoided capacity cost is large (impact model)

Together these support one simple test: **cost of flexibility < value of getting power sooner.**

Future scope:

1. **Deal pricing:** the minimum incentive a data center needs, in cash ($ per kW-year), per MWh cut, or months of earlier connection. Compare it with the most a grid could pay, its avoided capacity cost; the overlap is where a deal works for both.
2. **Negotiation offers:** "75 MW for free", "100 MW at $X per kW-year", "100 MW if connection moves 18 months earlier".
3. **Local grid limits:** substation and line constraints, using utility hosting-capacity data.
4. **Spatial flexibility:** moving work between data centers on different grids.
5. **Marginal emissions and real-time prices** in the optimizer.
6. **More regions:** PJM, MISO and other grids with growing data center demand.

Build plan

Start from the existing prototype (`flex-calculator-poc.zip`). It already has the headroom sweep, the forecaster, the LP scheduler and a Streamlit app, all on a synthetic grid. Today's work:

- swap in real ERCOT data
- add the day-by-day replay and reliable MW
- add the design search
- add the impact model

What ships, in priority order

Tier	Scope	Tests covered
Must ship	Real ERCOT data with carbon intensity (M0); grid capacity (M1); forecaster with benchmarks (M2); scheduler (M3); replay (M4, actual demand if forecasts aren't wired in yet); reliable MW (M5); impact model for grid, data center and emissions (M7); deployed app (M8)	A1–A3, A6, A7, A9, A10
Should ship	Design search on a reduced grid, e.g. 3 sizes × 4 batteries (M6); replay driven by forecasts; reliability re-test on summer 2025; 4CP check; ratepayer metrics	A4, A5, A8
Stretch	Carbon-price scheduling; real prices; ±2 °C noise run; mixed-integer design search; PJM	—

Team split (4 people)

Role	Owns	Hands over
Data engineer	M0 (including carbon intensity), `build.py` , deployment	`grid_hourly.parquet` early, so everyone else can switch from synthetic data
ML lead	M1, M2	`headroom.csv` , `forecast_test.csv` , `metrics.json`
Optimisation lead	M3, M4, M5, M6	`replay_hourly.csv` , `events.csv` , `reliability.json` , `scenarios.parquet`

Role	Owns	Hands over
Product and pitch lead	M7, M8, slides, video	`impact.json`, deployed URL, 3-minute script, business case slide

Checkpoints

This assumes about 8 working hours to the deadline; scale to your actual cut-off.

1. **Hour 2:** real ERCOT parquet exists; the impact model runs on the prototype's synthetic replay.
2. **Hour 4:** headroom checked against Duke; forecaster scored; replay runs on the real summer.
3. **Hour 6:** reliable MW and impact numbers are real; all five tabs work; the app is deployed. Freeze the must-ship tier.
4. **Hour 7:** design search and forecast-driven replay, if they're green. Record the backup video.
5. **Hour 8:** demo rehearsed twice; submit.

Risks and fallbacks

- **EIA API problems** → use the six-month CSVs or PUDL.
- **Forecast-driven replay not ready** → replay on actual demand (perfect foresight) and say so; the forecast is still scored on its own.
- **Design search too slow** → shrink the grid, or run on one month of the summer.
- **Deployment fails** → run locally, and show the recorded video plus screenshots.
- **Running out of time** → cut stretch, then the design search. Never cut the replay or the impact tab.

Demo flow (3 minutes)

1. **The grid (30 s):** "ERCOT is above this line for only about 40 hours a year. Here's how many GW fit if new load can flex."
2. **The forecast (30 s):** "We predicted those hours a day ahead, including all four of last summer's system peaks."
3. **The operation (45 s):** "Customer-facing AI never stops. Batch shifts, the battery covers short peaks, and training slows only at the top. The grid stays under its peak every hour."
4. **The design (30 s):** "This is the cheapest battery that keeps the promise 95% of the time, and the trade-off between size and compute lost."

5. **The impact (30 s):** walk the scorecards for capacity avoided, compute lost, net CO_2 and cost per MWh for everyone. Move one slider live.
6. **Close (15 s):** "That makes the business case: flexing costs less than waiting for power. Pricing that deal is our next step."

Submission checklist

- [ ] Public GitHub repo with README: setup, data sources, how to rebuild
- [ ] Frozen data snapshot committed, so the app runs offline
- [ ] Deployed app URL
- [ ] `metrics.json`, `reliability.json` and `impact.json` covering the acceptance tests
- [ ] 2-minute demo video
- [ ] 6-slide deck: problem, how it works, forecast, optimization, impact, business case and future scope
- [ ] Assumptions slide

Sources

- About the EIA-930 data: EIA
- EIA-930 in PUDL: Catalyst Cooperative
- Introducing the Historical Forecast API: Open-Meteo
- Duke researchers: grid flexibility key to accommodate load growth: POWER Magazine
- Texas law gives grid operator power to disconnect data centers during crisis: Utility Dive
- 4CP for Bitcoin miners: how one hour sets a year of ERCOT transmission costs: Hashrate Index
- Texas' peak game is getting harder as AI load floods ERCOT: The Energy Mag
- What is demand response?: Hashrate Index
- PJM capacity auction prices clear at record highs: S&P Global
- GridCARE and Portland General Electric: Utility Dive
- Emerald AI raises $150M: SiliconANGLE